INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Mutual Exclusion

Source: Distributed Computing by Kshemkalyani and Singhal

MTE Answers

Q1 (a) State the difference between Migration and Relocation transparency.

Migration: Hide that a resource may move to another location

Relocation: Hide that a resource may be moved to another location while in use

ISO 1995

Q1 (b) Flynn's Taxonomy categorizes computer systems into four types. What are they?

- Single Instruction Single Data (SISD),
- Single Instruction Multiple Data (SIMD),
- Multiple Instruction Single Data (MISD),
- Multiple Instruction Multiple Data (MIMD)

IIT ROORKEE

2

MTE Answers

Q1 (c) What properties a reflexive, weak, or non-strict partial order must satisfy?

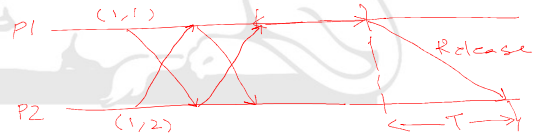
Is happened before relation "→" a reflexive, weak, or non-strict partial order? Justify your answer.

- It should be reflexive (a ≤ a), antisymmetric (if a ≤ b and b ≤ a, then a = b) and transitive (if a ≤ b and b ≤ c then a ≤ c).
- No, happened before relation "→" is irreflexive, strong, or strict partial order. It is irreflexive as a → a is not true.

MTE Answers

Q1(d) Assume that a message takes time T to reach destination from source. Sites send REPLY immediately on receiving a REQUEST. What is the synchronization delay under heavy load conditions if the sites are using Lampot's algorithm for mutual exclusion? Justify your answer.

- T. Let us assume that site 1 is in critical section (CS) and site 2 is waiting for 1 to release CS. When site 1 exits CS, it will send RELEASE message. Site 2 will enter CS when it receives this message which is after time T.



MTE Answers

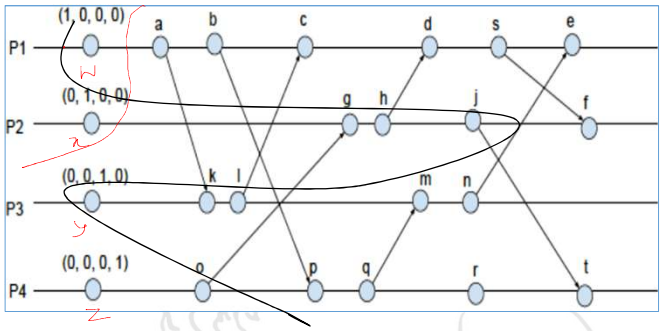
Q2 Write all pairs of concurrent events and explain for at least one pair that why the two events in the pair are concurrent.

e1 || e3 and e2 || e3

e1: (3, 1, 5, 7)

e3: (2, 1, 6, 8)

MTE Answers



Q3 (a) show the consistent cut that includes event **j** and the **least possible** number of other events.

x, g, h, j, z, o

(b) Find the vector time of the cut in (a) above.

1513 (cut is an event) or 0402 (cut is not an event)

MTE Answers

Q4 Maximum error (ϵ) in the estimation of time by daemon

$$\epsilon = \frac{T_M - 2 \min (T_{mAB}, T_{mBA})}{2} \geq 0.$$

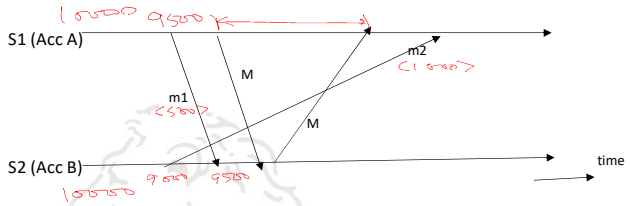
where, T_M = is the bound on RTT

($T_M \geq 2*\max(T_{mAB}, T_{mBA})$ and $RTT \leq T_M$)

T_{mAB} = minimal possible transmission times from ***A*** to ***B***

T_{mBA} = minimal possible transmission times from ***B*** to ***A***

MTE Answers



Q5 (a) Global state: S1: <9500>, S2: <9500>, c1: < >, c2: < >

(b) No, this global state is NOT consistent. This is because channels are not FIFO.

MTE Answers (Q6)

```
int main(){
        short port = 8450, port2;
        char *p = (char *) &port;
        char *p2 = (char *) &port2;

        for (int i = 0; i < sizeof(port); i++)
        printf("%x ", (unsigned char) *p++);
        printf("\n");
        port2 = htons(port);
        for (int i = 0; i < sizeof(port2); i++)
                printf("%x ", (unsigned char) *p2++);
}
```

$8450 = 8192 + 256 + 2 = (0010\ 0001\ 0000\ 0010)_2$

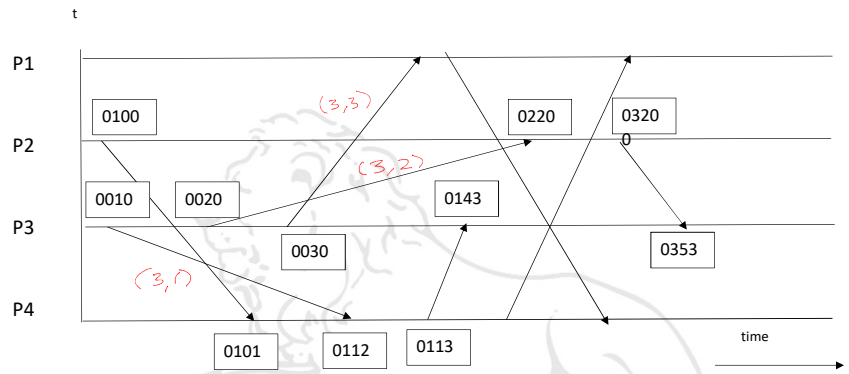
Output: 2 21

21 2

IIT ROORKEE

9

MTE Answers (Q7)



- Compressed timestamp: First message – (3, 1); Second message – (3, 2); Third message – (3, 3)
- LS_3[]: [00-1]; [02-1]; [32-1]; [32-1]; [32-1]
- LU_3[]: [0010]; [0020]; [0030]; [0444]; [0554]

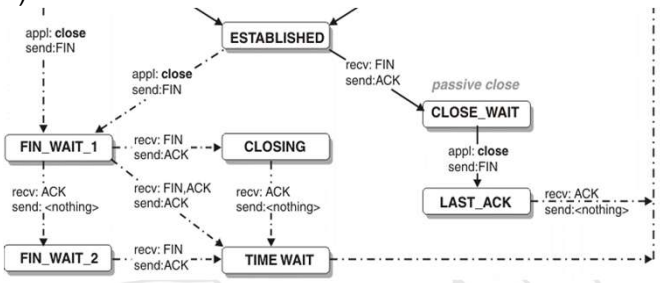
IIT ROORKEE

10

```
int main(int argc, char ** argv){
  /* variables declared here - not shown */
  ...
  listenfd = socket(AF_INET, SOCK_STREAM, 0);
  bzero(&servaddr, sizeof(servaddr));
  servaddr.sin_family = AF_INET;
  servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
  servaddr.sin_port = htons(6500);
  bind(listenfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
  listen(listenfd, 5);
  for(;;) {
    clilen = sizeof(cliaddr);
    connfd = accept(listenfd, (struct sockaddr *) &cliaddr,
&clilen);
    if ((childpid = fork()) == 0) {
      close(listenfd);
      n = read(connfd, str, 512);
      write(connfd, str, n);
      close(connfd);
      exit(0);
    }
  }
}
```

MTE Answers (Q8)

- Because server parent process has not closed connfd descriptor. TCP on server side will not initiate connection termination (server TCP will not send FIN)




```
Proto Recv-Q Send-Q Local Address    Foreign Address          State
tcp        0      0 *:6500           *:*                      LISTEN
tcp        1      0 localhost:32778  localhost:6500           FIN_WAIT_2
tcp        0      0 localhost:6500     localhost:32778        CLOSE_WAIT
```

Ricart-Agarwala Algorithm

- Idea:

  –node j need not send a REPLY to node i if j has a request with timestamp lower than the request of i

  (since i cannot enter before j anyway in this case)

- requests granted in order of increasing timestamps
- Request-deferred array RD_i of size n where n is the number of processes
- Initially for all i, j: $Rd_i[j] = 0$

To request critical section:

- send timestamped REQUEST message (ts_i, i) to all

On receiving request (ts_i, i) at j:

- send REPLY to i if j is neither requesting nor executing critical section or **if j is requesting and i's request timestamp is smaller than j's request timestamp.**
- Otherwise, defer the request and set $RD_j[i] = 1$

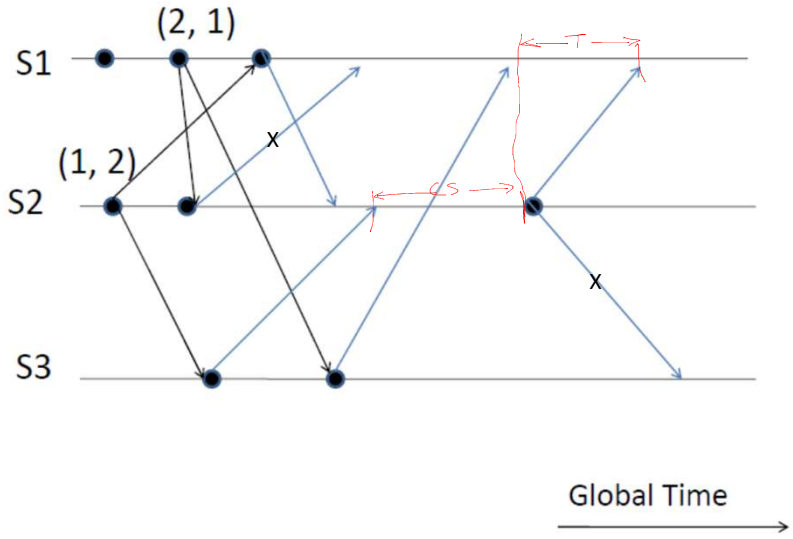
To enter critical section:

- i enters critical section on receiving REPLY from all nodes

To release critical section:

- send REPLY to all deferred requests and set $RD_i[j] = 0$

Example: Mutual Exclusion



Ricart-Agarwala Algorithm

- A site's REPLY messages are blocked only by sites that are requesting the CS with higher priority
- Ricart-Agarwala algorithm **achieves mutual exclusion**

Proof by contradiction

Assume both i and j in CS and (tsi, i) < (tsj, j).

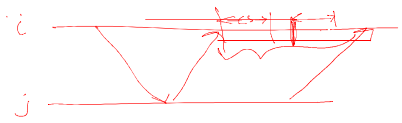
⇒ i received j's request after its own request (otherwise i's timestamp should have been larger)

⇒ j can enter CS only after getting i's reply – as per algorithm i will not send reply if j's timestamp is higher

Ricart-Agarwala Algorithm

- Ricart-Agarwala algorithm is fair
- 2(n – 1) messages per critical section invocation
- Synchronization delay = max. message transmission time
- Improvement – 0 to 2(N-1) messages per CS execution by Roucairol and Carvalho

Ricart-Agarwala Algorithm



- Improvement

  once a node i has received a REPLY message sent by a node j, the authorization implicit in this message remains valid until i has decided to send a REPLY message to j.

  This decision happens only after the reception of a REQUEST message coming from j; during this interval, node i will be able--as far as j is concerned--to enter its critical section more than one time without consulting node j.

  - Roucairol and Carvalho

Ricart-Agarwala Algorithm

Each node has three processes to implement the mutual exclusion:

1. One is awakened when mutual exclusion is invoked on behalf of this node.
2. Another receives and processes REQUEST messages.
3. The last receives and processes REPLY messages.

```
CONSTANT   me,      ! This node's unique number
           N;       ! The number of nodes in the network
INTEGER    Our_SequenceNumber,
                    ! The seq number chosen by a request
                    ! originating at this node
           HighestSequenceNumber initial (0),
                    ! The highest sequence number seen in any
                    ! REQUEST message received
           Outstanding_Reply_Count;
                    ! The number of REPLY messages still
                    ! expected
BOOLEAN    Requesting Critical_Section initial (FALSE),
                    ! TRUE when this node is requesting access
                    ! to its critical section
           Reply_Deferred [1:N] initial (FALSE);
                    ! Reply_Deferred [j] is TRUE when this
                    ! node is deferring a REPLY to j's REQUEST
```

PROCESS WHICH INVOKES MUTUAL EXCLUSION FOR THIS NODE

```
// Request Entry to our Critical Section;
 P (Shared_vars)
        Requesting_Critical_Section := TRUE;
        Our_Sequence_Number := Highest_Sequence_Number + l;
 V (Shared_vars);
 Outstanding_Reply_Count := N - l;
 FOR j := 1 STEP l UNTIL N DO
  IF j != me THEN
   Send_Message (REQUEST(Our_Sequence_Number, me), j);
    // REQUEST message to all containing our seq  & node number
   WAITFOR (Outstanding_Reply_Count = 0);
    //Critical Section Processing can be performed at this point;
```

PROCESS WHICH RECEIVES REQUEST (k, j) MESSAGES

```
// k is the sequence number j is the node number making the request;
BOOLEAN       Defer_it ; ! TRUE when we cannot reply immediately
Highest_Sequence_Number := max (Highest_Sequence_Number, k);
 P (Shared_vars);
    Defer it := Requesting_Critical_Section AND ((k >
    Our_sequence_Number) OR (k = Our_Sequence_Number AND j >
    me));
 V (Shared_vars);
// Defer_it will be TRUE if we have priority over node j's request;
    IF (Defer it) THEN Reply_Deferred[j] := TRUE
    ELSE Send_Message (REPLY, j);
```

PROCESS WHICH RECEIVES REPLY MESSAGES

```
Outstanding_Reply_Count := Outstanding_Reply_Count - 1;
```

PROCESS WHICH INVOKES MUTUAL EXCLUSION FOR THIS NODE

```
// Release the Critical Section;
  Requesting_Critical_Section := FALSE;
  FOR j := l STEP 1 UNTIL N DO
  IF Reply_Deferred [j] THEN
  BEGIN
    Reply_Deferred[j] := FALSE;
    Send_Message (REPLY, j);
    //send a REPLY to node j;
    END;
```